Crew 1.1 - the Full Guide

This guide explains the whole of crew 1.1 in one place: what it is, how a ticket moves from an idea to a merged change, what the guards can and cannot see, and what to do when something goes wrong. The five task guides go deeper on each part, and each section below links to the one that does. Every setting is listed in the separate configuration reference; see Where the settings are.

Each section starts with the command or file it describes. A feature that has not landed carries the ticket that brings it, and is listed in What is coming.

What Crew Is, and the Four Roles

Source: `plugin/crew/README.md`, sections 1 and 22.

crew is a workflow, not an org chart. A ticket is a set of files, one interactive session implements it, an independent reviewer reads the finished change, and deterministic gates block on failure instead of giving an opinion.

crew 1.1 ships four agents. Each one exists because it buys something a prompt cannot: an isolated context window, a restricted tool set, or independent eyes.

Role	What it does	What it cannot do
`session-relay`	Carries messages between sessions on Haiku 5.5; transport only, not a role	No Write, Edit or Bash; SendMessage, ListAgents, Read
`explorer`	Maps the code a ticket touches	No Write or Edit; Bash only to hand its task to Kimi
`reviewer`	The Claude rung of the review order, used when Codex or Kimi is not available	Writes nothing to the tree
`security`	Read-only review of risky changes	No Write or Edit
`researcher`	Reads external sources only	Does not edit the repo

Running agents on Kimi. Claude Code runs subagents on Claude models only, so `explorer`, `researcher` and `security` can hand their task to the Kimi Code CLI instead. Pin one with

`/crew:model dev.roles.explorer '{"provider": "kimi", "model": "k3"}'`. Kimi then answers read-only, and the agent falls back to Claude when Kimi cannot run. Its first line says which one ran. A Kimi `security` review is refused on code Kimi wrote, because the family that wrote a change may not judge it. Nothing is pinned until you pin it.

The session you type into does the implementing. Nothing else is dispatched to write. Stack knowledge (Python, Bash, PowerShell, Terraform and others) loads on demand as `stack-*` skills. There is no PM agent, and nothing grows or shrinks the roster. `/crew:status` reports the roster read-only.

Parallel work scales with independent units, not job titles. Two repositories, or two git worktrees of one repository, run in parallel. Two agents in one working tree do not: their edits conflict.

Install and Update

Source: `README.md` (the marketplace install) and `plugin/crew/README.md`, section 4.

Install crew from this marketplace once (`claude plugin marketplace add mbadali25/useful-claude-add-ons`, then `claude plugin install crew@useful-claude-add-ons`), and update it in two steps:

```
claude plugin marketplace update useful-claude-add-ons

claude plugin update crew@useful-claude-add-ons
```

Then **restart** `claude`. A running session keeps the copy it loaded. In a session that is already open, `/reload-plugins` picks up a changed agent, hook or MCP config without a restart.

`claude plugin update` compares the **declared version**, not the content. A release that changed files without bumping `version` reports "already at the latest version" on every machine that installed the previous copy, and keeps the old code. If you can see a fix in the repository and your copy does not have it, that is a release defect, not something a retry fixes.

Deeper: Quickstart, and "Stale install" in Troubleshooting.

Init and Migrate

Source: `plugin/crew/commands/init.md` and `plugin/crew/commands/migrate.md`.

`/crew:init` sets a repository up: it writes `.crew/config.json`, picks a tracker, and records the platform. It writes every key of the repo template, and a `null` it writes never shadows a value you set in your machine-global file. The template leaves out the five personal `autopilot.*` settings, so your machine-wide defaults for them apply. For a new config it sets `scope.mode` to `auto`: the scope guard reports for the first ten approved tickets, then blocks.

Init also keeps `.gitignore` right for the languages it finds: `crew_gitignore.py apply` adds the missing patterns, without asking, inside one `# crew:gitignore:managed` block at the top of `.gitignore`, and never edits your own lines or untracks a file. A committed secret-shaped file is reported to you instead (exit 3). `/crew:onboard` and `/crew:onboard --refresh` run the same step, `/crew:implement` checks it, and `/crew:status` shows it as one `gitignore` line.

`/crew:init` writes the repo config to `.crew/config.json`, the file every gate, guard and `/crew:config` reads and writes; a new repository needs nothing else. `/crew:migrate` is only for a repository crew 0.20 set up (`/crew:status` says `run /crew:migrate`): it moves the tickets, metrics and PM journal to the 1.0 layout and writes `.crew/crew.json` as a record of the old config, once per repository: `--preview` first, then `--apply`, which takes a backup that `--rollback` restores. `.crew/config.json` stays, and every gate, guard and `/crew:config` keeps reading it; the context hook, resume and the diagram commands read `.crew/crew.json` first when it exists, so after a migrate a `memory`, `context` or `docs` setting is changed in both files. Every historical metric it cannot recover is written `UNKNOWN`, never `0`. A config older than 0.20 (no `schema`, or 1-6) is upgraded by `/crew:migrate` itself in the same run; `/crew:upgrade` was removed.

Deeper: Quickstart.

The Lifecycle

Source: `plugin/crew/hooks/scripts/crew_ticket.py` and `plugin/crew/commands/approve.md`.

A ticket is a directory under `.work/tickets/` holding three files:

- `direction.md`, written by `/crew:brainstorm`: the problem and the direction you agreed, one question at a time.
- `spec.md`, written by `/crew:spec`: the contract, with six sections (Intent, Exclusions, Evidence, Unknowns, Touch, Acceptance checks). `## Touch` lists the paths the ticket may change.
- `plan.md`, written by `/crew:plan`: the steps, each with its files, test, risk and standards.

Then:

1. **Approve.** You type `/crew:approve <id>` yourself. A hook reads `spec.md` and `plan.md`, validates them, and writes an approval receipt holding their hashes. A contract that does not validate blocks the prompt and records nothing.
2. **Implement.** `/crew:implement <id>` runs the plan, the tests, the docs and the refresh check, then review.
3. **Review.** `/crew:review` (next section).
4. **Done.** `/crew:done` checks the review receipt, the gates and the refresh artifacts, then closes the ticket.

Why an edit stales the approval. The receipt binds the bytes you approved. Change `spec.md` or `plan.md` (to widen Touch, say) and the approval is `stale` until you approve again. One edit is allowed: the value of the header's `status:` token, so the commands can move a ticket through `spec`, `planned`, `review` and `done` without a new approval. `crew_ticket.py status --ticket <id>` prints `approved`, `stale` or `none`.

Several tickets can be approved together. `/crew:approve T-4..T-6` shows a pending list first, and `/crew:approve --confirm` records them, all or none.

Deeper: Daily workflow and Daily workflow: scope and approval.

Review, the Ledger and Acceptance

Source: `plugin/crew/hooks/scripts/review_ledger.py` and `plugin/crew/commands/review.md`.

`/crew:review` sends a bundle of the whole change to Codex or Kimi when one is available, and to the `reviewer` agent when not. It always says which ran. A large bundle is split into ordered parts, never truncated. A script, not the reviewer, computes the verdict:

- `CLEAN`: one `CLEAN` line and every part acknowledged. It writes an acceptance receipt bound to the bundle's hash.
- `FINDINGS`: `BLOCK`, `FIX` or `NIT` lines.
- `INCOMPLETE`: anything else, such as a crash, a timeout or a part not read. It is never `CLEAN`.

Two rounds per ticket, in total. The ledger lives under the repository's git common directory, so every worktree shares it. A round is reserved before the reviewer starts, so a crash still

spends it. A round the tool lost (a timeout, or no answer) is refunded, at most twice per plan. A third reservation is refused and the ticket becomes `NEEDS_REPLAN`.

Successor plans. The way past `NEEDS_REPLAN` is a different plan: approving a changed `plan.md` opens a fresh budget of two rounds. Approving the same plan again does not.

Acceptance is yours. `FINDINGS` become a receipt only when you run `review_ledger.py --ticket <id> --accept --by <you>`. It refuses when the tree changed since that round. One exception: when the final round allowed is `FINDINGS` with no `BLOCK`, from a reviewer of a different model family, the ledger accepts it automatically and files one follow-up ticket quoting every finding. Any edit after review invalidates the receipt; committing the reviewed change does not.

Deeper: "Review loops" in Troubleshooting.

Refresh Artifacts

Source: `plugin/crew/commands/implement.md` and `plugin/crew/hooks/scripts/crew_refresh_check.py`.

A refresh artifact is a file that describes the code and must follow it: the code maps under `.crew/codemap/`, the diagrams, the code graph in `graphify-out/`, the generated `.claude/rules/`, and the integrations reference (docs/reference/integrations.md) that `/crew:reference --integrations` writes. The order is fixed: implement, then refresh, then review, then done.

`crew_refresh_check.py --root . --ticket <id>` reports each artifact the ticket's changes reach as `fresh`, `stale` (with the command that refreshes it: `/crew:onboard --refresh <subsystem>`, `/crew:diagram refresh`, `/crew:reference --integrations` or `/crew:graph --refresh`) or `unknown`. `/crew:implement` runs those commands and commits before review, so the reviewer reads the refreshed files. `/crew:done` runs the same check and refuses anything but `fresh`. It never refreshes anything itself: a write after review would stale the receipt.

The Guards

Source: `plugin/crew/hooks/scripts/scope_guard.py`, `plugin/crew/hooks/scripts/completion_audit.py`, `plugin/crew/hooks/scripts/cloud_guard.py`, `plugin/crew/hooks/scripts/env_guard.py` and `plugin/crew/hooks/scripts/promote-gate.sh`.

What is on by default, and what is not:

- **Off until you arm it:** the scope guard (`scope.mode` is `off` unless `/crew:init` set it), the cloud guard's hook (`guards.cloudGuard: off`), the environment guard (`guards.envGuard: off`) and the role-write guard (`guards.roleWrites: off`).
- **`block` by default:** the six policy guards (`guards.terraformApply`, `forcePush`, `adminMerge`, `mergeGate`, `cloudDestructive`, `sqlDestructive`) and the two production guards (`prodDatabase`, `prodServer`, which default to `none`). The policy guards take effect only once `guards.cloudGuard` is armed, except `mergeGate`, which `/crew:gate` reads.
- **On by default:** the Stop verify gate (`verifyGate: true`) blocks when a verify rule fails.

The scope guard (`scope.mode`) runs before each Write, Edit, MultiEdit and NotebookEdit. It refuses an edit when the active ticket has no current approval, or when the target is outside Touch. It also refuses a shell command that tries to write the approval state. *What it cannot see:* `sed -i`, a redirect or a formatter writing a file. Those are shell writes, and the guard judges only the editing tools.

The completion audit runs at Stop. It diffs the whole tree against the ticket's start (committed, staged, unstaged and untracked), so shell-made writes are caught after the fact. *What it cannot see:* gitignored files (`.crew/*` among them) and `.work/`.

The cloud guard (`guards.cloudGuard`, default `off`) judges Bash and PowerShell commands: `terraform apply`, a force push, an admin merge, destructive cloud CLI calls, destructive SQL, and anything aimed at a declared production database or host. Each rule has its own `guards.*` setting. Set `report` first to see what `block` would cost. *What it cannot see:* a command hidden in an interpreter (a Python script that calls the cloud SDK), a variable or an encoded string. The credentials boundary that would close that is T-0044.

The environment guard (`guards.envGuard`, default `off`; L-0772) refuses a Bash or PowerShell command whose own text prints the whole process environment (`env`, `printenv`, bare `set`, `export -p`, `cat /proc/self/environ`, `Get-ChildItem env:`) or a credential-named variable (`echo "$GITHUB_TOKEN"`, `$env:API_KEY`) into the agent's context, where it would land in the transcript. A command it cannot read is refused as could-not-tell. Set `report` first. *What it cannot see:* a script or program the command runs, an alias defined outside the command, or a renamed binary; keeping credentials out of the agent's environment is the stronger control (L-0776).

The promote gate runs on a deploy command declared in `.crew/verify.json`. Before the deploy it checks that the upstream environment passed for this exact sha (a promotion row carries the full sha), that a rollback is declared, that a person approved it when `requireHuman` is set, that an accepted review covers the exact tree being deployed (unless the environment opts

out with `requireReview: false` and a reason), and that the tree is clean. The review evidence proves only that the tracked tree deployed is one a reviewer was shown under a standing receipt: it does not prove that paths the review bundle left out (identical to merged main, or under `.work/`, `graphify-out/`, `.crew/metrics.md`) or ignored build output a deploy script ships were reviewed, nor who wrote the local ledger.

The guards stop drift and accidents, not a session set on forging local state: the receipts are files on your machine. The completion audit and the reviewer are the backstop.

Deeper: Daily workflow: scope and approval, and "Cloud guard false positives" in Troubleshooting.

Autopilot Today

Source: `plugin/crew/commands/autopilot.md` and `plugin/crew/hooks/scripts/crew_autopilot.py`.

`/crew:autopilot <id>` drives one ticket through spec, plan, approval, implement, refresh, review and done. Each turn it asks `crew_autopilot.py next` which phase comes next, read from files on disk, so a skipped phase is visible. It is off until `autopilot.mode` is exactly `plan` (one ticket per run) or `backlog`. `/crew:autopilot --goal <slug>` works a goal's minted tickets one at a time in dependency order, each through its own approval; `backlog` goes on to the next ticket while `autopilot.maxTicketsPerRun` (3) and `autopilot.maxTokensPerSession` (2,000,000) allow, and every stop prints `resume: /crew:autopilot --goal <slug>` (L-0541).

The five `autopilot.*` settings are personal: set them in the repo's `.crew/config.json`, or once in the machine-global file as your default for every repo. Where both files set one, the stricter value wins, so a repo can narrow your default and never widen it.

What always stops for a person, at any setting:

- the brainstorm, which is a dialogue with you;
- accepting review findings that include a `BLOCK`;
- a review ledger at `NEEDS_REPLAN`, or anything autopilot cannot tell.

Plan approval and open questions stop too, unless `autopilot.approval` and `autopilot.questions` allow otherwise. Plan approval also needs `scope.allowCliApproval: true` in the repo file, even under a machine-wide `autopilot.approval: self`; an open question does

not.

After done, autopilot ships the ticket (since crew 1.0.349). `autopilot.ship` set to `pr` pushes the branch and opens a pull request, then stops for a person to merge. Set to `merge`, the default, it also merges once every required check passes, or fails under a name listed exactly in `autopilot.knownFailures`, waiting up to `autopilot.ciTimeoutMinutes`. It always merges with a merge commit, never a squash, a rebase or `--admin`, and never through a merge queue. A `high`-risk ticket, or one whose spec names no risk, never merges unattended when every review was same-family (Claude reviewing Claude); it stops with its pull request open. Autopilot never deploys today. Goals and backlogs arrive with T-0012, and deploy dispatch with T-0045. `/crew:autopilot status` prints where a ticket stands, read-only.

`/crew:autopilot focus <id>` locks autopilot onto one ticket. Focus is on only after you type it: an active ticket on its own is not a focus. It writes a focus marker for this worktree (and points the active ticket at `<id>` if it named another, printing the scope-base line). While focused, autopilot refuses to run another ticket, `assign` and `goal`; `status`, `sleep` and `wake` still run. Once the plan is approved, `next` stops as `drift` on a changed path outside the plan's Touch list, or when that check could not run. A focus marker that cannot be read counts as "could not tell", and autopilot refuses new work until you repair or remove it. `focus --findings --ticket <id>` names where an out-of-scope finding goes. `/crew:autopilot focus` shows the lock and `/crew:autopilot focus off` releases it. Claude Code's built-in `/focus` is unrelated: it only toggles the display.

A separate autopilot guide with worked examples is planned as T-0054.

Trackers

Source: `plugin/crew/hooks/scripts/crew_tracker.py` and `plugin/crew/commands/obsidian-sync.md`.

The `tracker` setting says where tickets live: `files` (the default: `.work/INDEX.md` and the ticket folders), `obsidian` (a Kanban board in a vault), `jira` or `sdp` (ServiceDesk Plus). `crew_tracker.py` resolves it, and answers `could not tell` when two config files disagree, never a guess. `/crew:obsidian-sync`, `/crew:jira-sync` and `/crew:sdp-sync` push ticket state to the board.

Deeper: Memory and Obsidian.

Multi-Session Work

Source: `plugin/crew/hooks/scripts/crew_train.py` and `plugin/crew/README.md`, section 10.

Several sessions can work one repository at once, each in its own git worktree:

- **One active ticket per worktree.** `crew_ticket.py activate --ticket <id>` sets it, and the scope guard and the audit judge each worktree's edits against its own ticket.
- **Shared state lives in the git common directory**: the approval receipts, the review ledger and the merge train's queue. Every worktree of a clone sees the same review budget.
- **The merge train** (`crew_train.py`, armed per clone) serialises the gate-and-land stage for tickets whose Touch lists overlap. Catch-up is always a merge, never a rebase. The train never merges, pushes or opens the PR; only autopilot's ship step does, under `autopilot.ship`.
- **Check before assuming a diff is yours.** Another session may have changed the tree, a branch or the stash (root `CLAUDE.md`, "Lessons").

Cross-session coordination claims arrive with T-0030.

Windows

Source: `plugin/crew/skills/crew-setup/platform.md` and `CLAUDE.md`, "Landmines".

- Every hook is registered twice, once for bash and once with `shell: "powershell"`. Each twin stands down on the wrong platform.
- `pwsh` is not on Git Bash's `PATH`, and Git Bash has no `python3`. crew's wrappers resolve both explicitly and fail loudly when neither resolves.
- Long jobs follow `shellRoute.mode` (`auto`, `wsl`, `powershell` or `gitbash`). `/crew:status`'s `shell` line says which route is in force and why.
- Auto-clear on Windows types only with `context.autoClear.method: "sendkeys"`, and only after checking the window has focus.

Deeper: "Windows notes" in Troubleshooting and Auto wrap-up, clear and resume.

Troubleshooting

Source: `CLAUDE.md`, "Landmines", and `docs/guides/crew/src/troubleshooting.md`. Each entry is a real failure from this repository.

A fix is in the repository but not on your machine. Cause: the release changed content without bumping `version`, so `claude plugin update` sees nothing new. Fix: bump the version in every place `scripts/check-marketplace.py` checks, release again, then update.

`pwsh: command not found` from Git Bash. Cause: PowerShell is not on Git Bash's `PATH`. Fix: name it by its absolute path. The gate reports this as a failed check, not a missing tool.

A hook says nothing on Windows. Cause: Git Bash has no `python3`. Fix: install Python, or put `python` or `py` on `PATH`. crew's wrappers try all three.

A shell script fails with `$'\r': command not found`. Cause: a Python writer created the `.sh` file with CRLF line endings on Windows. Fix: write with `newline="\n"`, then restore the file with `git checkout -- <path>`.

A path argument arrives as `C:/Program Files/Git/...`. Cause: Git Bash rewrites arguments that look like POSIX paths. Fix: run the command with `MSYS_NO_PATHCONV=1`.

The gate fails on `plugin/crew/BUDGETS.md`'s line count. Cause: any edit to a `plugin/crew/*.md` file moves the total that `plugin/crew/BUDGETS.md` states, and a ticket whose Touch omits it cannot re-measure it. Fix: add it to Touch and re-measure. T-0046 plans to admit release bookkeeping like this without a Touch entry.

A background agent dies part-way. Possible cause: something outside the work stopped it, such as a usage or session limit on the account. Fix: check what it committed, then resume from the handoff rather than starting again.

`pwsh` crashes on start with `Stack overflow.` Cause: a corrupt `StartupProfileData-NonInteractive` file in `~/.cache/powershell`. The leading explanation, not yet confirmed, is a torn write: several pwsh processes (a snap install, here) rewriting that one file at once. Fix: move that file aside; pwsh rebuilds it on the next start. crew's own test suites now give every pwsh they spawn a throwaway `XDG_CACHE_HOME`, so they cannot corrupt it (`plugin/crew/README.md`, section 25). Ticket T-0506 tracks the cause.

`--accept` refuses: "the tree has changed since that review". Cause: you ran `review_ledger.py --accept` from a checkout whose tree is not the one reviewed, often the main checkout instead

of the ticket's worktree. Fix: run it from the worktree that holds the reviewed change.

A new hook or command does not appear after an update. Cause: the running session cached the old plugin. Fix: `/reload-plugins`, or restart `claude`.

Deeper: Troubleshooting.

What Is Coming

These tickets are approved or filed and have not landed. Nothing in this guide describes them as present.

Ticket	What it brings
T-0009	A ratcheted guard for deploy workflows, and per-environment deploy workflows
T-0012	Autopilot goals and backlogs, with per-run caps
T-0019	`/crew:autopilot assign`
T-0029	Parallel autopilot lanes and a review policy
T-0030	Cross-session coordination claims
T-0044	A credentials boundary for what the cloud guard cannot see
T-0045	Autopilot deploy dispatch (the `autopilot.deploy` setting exists and is inert until then)
T-0046	Release bookkeeping admitted without a Touch entry
T-005	A separate autopilot guide with worked examples

Ticket	What it brings
4	
T-0506	The cause of the pwsh profile-cache crash

The configuration reference lists the settings each of these adds, under "Coming".

Where the Settings Are

crew reads two files: the machine-global `~/.claude/crew/config.json` and the repo's `.crew/config.json`. The repo file wins, except where a setting ratchets, is personal (the stricter of the two wins), or only the machine file may arm it.

- `/crew:config` shows where every value comes from, and sets either file through a validated, dry-run-first menu.
- The configuration reference lists every setting with its allowed values, default, layer and the version it arrived in. It is generated from the code.
- `plugin/crew/CONFIG.md` holds the reasoning behind each setting, and ships with the plugin.

Every write a crew script makes to either file first copies the old file to `~/.claude/crew/backups/` and is refused if that copy fails (a hand edit is not backed up); `crew_config.py --backups` lists the copies and `--restore <stamp>` puts one back. crew also keeps your non-default values in `~/.claude/crew/profile.json`, so `crew_config.py --rebuild --repo` (or `--global`) regenerates a lost or corrupt config from the template plus that profile, as a dry run until `--apply`.